

```

import { useState, useRef, useEffect } from
"react";

const BOT_AVATAR = () => (
  <svg width="32" height="32" viewBox="0 0
32 32" fill="none">
    <circle cx="16" cy="16" r="16"
fill="#6C63FF" />
    <rect x="9" y="11" width="14"
height="10" rx="3" fill="white"
opacity="0.95" />
    <circle cx="13" cy="15" r="1.5"
fill="#6C63FF" />
    <circle cx="19" cy="15" r="1.5"
fill="#6C63FF" />
    <rect x="14" y="8" width="4" height="3"
rx="1" fill="white" opacity="0.8" />
    <circle cx="16" cy="8" r="1"
fill="#A78BFA" />
    <rect x="7" y="14" width="2" height="4"
rx="1" fill="white" opacity="0.7" />
    <rect x="23" y="14" width="2"
height="4" rx="1" fill="white"
opacity="0.7" />
  </svg>
);

const TypingIndicator = () => (
  <div style={{ display: "flex",
alignItems: "center", gap: 4, padding:

```

```

"10px 14px", background: "#F3F2FF",
borderRadius: 18, borderBottomLeftRadius:
4, width: "fit-content" }}>
  {[0, 1, 2].map(i => (
    <span key={i} style={{
      width: 7, height: 7, borderRadius:
"50%", background: "#6C63FF",
      animation: "bounce 1.2s infinite",
      animationDelay: `${i * 0.2}s`,
      display: "inline-block"
    }} />
  ))}
</div>
);

const Message = ({ msg }) => {
  const isUser = msg.role === "user";
  return (
    <div style={{
      display: "flex",
      flexDirection: isUser ? "row-reverse"
: "row",
      alignItems: "flex-end",
      gap: 8,
      marginBottom: 16,
    }}>
      {!isUser && (
        <div style={{ flexShrink: 0 }}>
          <BOT_AVATAR />
        </div>
      )}
      <div style={{
        maxWidth: "72%",
        padding: "11px 15px",

```

```

        borderRadius: 18,
        borderBottomLeftRadius: isUser ? 18
: 4,
        borderBottomRightRadius: isUser ? 4
: 18,
        background: isUser ? "linear-
gradient(135deg, #6C63FF, #8B5CF6)" :
"#F3F2FF",
        color: isUser ? "white" :
"#1a1a2e",
        fontSize: 14,
        lineHeight: 1.55,
        boxShadow: isUser ? "0 2px 12px
rgba(108,99,255,0.25)" : "0 1px 4px
rgba(0,0,0,0.07)",
        whiteSpace: "pre-wrap",
    }}>
    {msg.content}
</div>
</div>
);
};

const SUGGESTIONS = [
    "What is machine learning?",
    "Explain neural networks simply",
    "How does NLP work?",
    "What can AI assistants do?",
];

export default function AIChatbot() {
    const [messages, setMessages] =
useState([
    { role: "assistant", content: "Hello!

```

```
👋 I'm your AI assistant. Ask me anything –  
I'm here to help!" }  
  });  
  const [input, setInput] = useState("");  
  const [loading, setLoading] =  
  useState(false);  
  const [showSuggestions,  
  setShowSuggestions] = useState(true);  
  const bottomRef = useRef(null);  
  const inputRef = useRef(null);  
  
  useEffect(() => {  
    bottomRef.current?.scrollIntoView({  
behavior: "smooth" });  
  }, [messages, loading]);  
  
  const sendMessage = async (text) => {  
    const trimmed = (text || input).trim();  
    if (!trimmed || loading) return;  
  
    setShowSuggestions(false);  
    const userMsg = { role: "user",  
content: trimmed };  
    const newMessages = [...messages,  
userMsg];  
    setMessages(newMessages);  
    setInput("");  
    setLoading(true);  
  
    try {  
      const apiMessages = newMessages.map(m  
=> ({ role: m.role, content: m.content }));  
      const response = await  
fetch("https://api.anthropic.com/v1/message
```

```

s", {
  method: "POST",
  headers: { "Content-Type":
"application/json" },
  body: JSON.stringify({
    model: "claude-sonnet-4-6",
    max_tokens: 1000,
    system: "You are a friendly,
helpful AI assistant. Keep responses
concise and clear – typically 2-4 sentences
unless more detail is genuinely needed. Be
warm and conversational.",
    messages: apiMessages,
  }),
});

const data = await response.json();
const reply = data.content?.map(b =>
b.text || "").join("") || "Sorry, I
couldn't get a response.";
setMessages(prev => [...prev, { role:
"assistant", content: reply }]);
} catch {
  setMessages(prev => [...prev, { role:
"assistant", content: "Something went
wrong. Please try again!" }]);
} finally {
  setLoading(false);
  inputRef.current?.focus();
}
};

const handleKey = (e) => {
  if (e.key === "Enter" && !e.shiftKey) {

```

```

        e.preventDefault();
        sendMessage();
    }
};

return (
    <div style={{
        fontFamily: "'Inter', -apple-system,
sans-serif",
        minHeight: "100vh",
        background: "linear-gradient(135deg,
#EEF2FF 0%, #F5F3FF 50%, #EDE9FE 100%)",
        display: "flex",
        alignItems: "center",
        justifyContent: "center",
        padding: 16,
    }}>
        <style>{`
            @keyframes bounce {
                0%, 60%, 100% { transform:
translateY(0); }
                30% { transform:
translateY(-6px); }
            }
            @keyframes fadeIn {
                from { opacity: 0; transform:
translateY(8px); }
                to { opacity: 1; transform:
translateY(0); }
            }
            .chat-input:focus { outline: none;
}

            .send-btn:hover { background:
linear-gradient(135deg, #5B52E0, #7C3AED)

```

```

!important; transform: scale(1.05); }
    .send-btn:active { transform:
scale(0.97); }
    .suggestion-chip:hover {
background: #6C63FF !important; color:
white !important; border-color: #6C63FF
!important; }
`}</style>

```

```

<div style={{
  width: "100%", maxWidth: 440,
  background: "white",
  borderRadius: 24,
  boxShadow: "0 20px 60px
rgba(108,99,255,0.15), 0 4px 16px
rgba(0,0,0,0.06)",
  overflow: "hidden",
  display: "flex",
  flexDirection: "column",
  height: "min(680px, 90vh)",
}}>
  {/* Header */}
  <div style={{
    background: "linear-
gradient(135deg, #6C63FF, #8B5CF6)",
    padding: "18px 20px",
    display: "flex",
    alignItems: "center",
    gap: 12,
  }}>
    <div style={{
      width: 42, height: 42,
borderRadius: "50%",
      background:

```

```

"rgba(255,255,255,0.2)",
      display: "flex", alignItems:
"center", justifyContent: "center",
    }}>
    <svg width="22" height="22"
viewBox="0 0 32 32" fill="none">
      <rect x="6" y="9" width="20"
height="14" rx="4" fill="white"
opacity="0.95" />
      <circle cx="12" cy="15" r="2"
fill="#6C63FF" />
      <circle cx="20" cy="15" r="2"
fill="#6C63FF" />
      <rect x="13" y="5" width="6"
height="4" rx="1.5" fill="white"
opacity="0.8" />
      <circle cx="16" cy="5"
r="1.5" fill="#C4B5FD" />
      <rect x="4" y="13" width="2"
height="6" rx="1" fill="white"
opacity="0.7" />
      <rect x="26" y="13" width="2"
height="6" rx="1" fill="white"
opacity="0.7" />
    </svg>
  </div>
  <div>
    <div style={{ color: "white",
fontWeight: 700, fontSize: 15,
letterSpacing: "-0.2px" }}>AI
Assistant</div>
    <div style={{ display: "flex",
alignItems: "center", gap: 5, marginTop: 2
}}>

```

```

        <span style={{ width: 7,
height: 7, borderRadius: "50%", background:
"#4ADE80", display: "inline-block" }} />
        <span style={{ color:
"rgba(255,255,255,0.8)", fontSize: 12
}}>Online</span>
    </div>
</div>
</div>

{/* Messages */}
<div style={{
    flex: 1, overflowY: "auto",
padding: "20px 16px 8px",
    display: "flex", flexDirection:
"column",
}}>
    {messages.map((msg, i) => (
        <div key={i} style={{
animation: "fadeIn 0.25s ease" }}>
            <Message msg={msg} />
        </div>
    ))}

    {loading && (
        <div style={{ display: "flex",
alignItems: "flex-end", gap: 8,
marginBottom: 16, animation: "fadeIn 0.2s
ease" }}>
            <BOT_AVATAR />
            <TypingIndicator />
        </div>
    )}

```

```

        {showSuggestions && !loading && (
          <div style={{ marginTop: 4,
marginBottom: 8 }}>
            <div style={{ fontSize: 11,
color: "#9CA3AF", marginBottom: 8,
textAlign: "center", textTransform:
"uppercase", letterSpacing: "0.5px" }}>
              Try asking
            </div>
            <div style={{ display:
"flex", flexWrap: "wrap", gap: 6,
justifyContent: "center" }}>
              {SUGGESTIONS.map((s, i) =>
                (
                  <button key={i}
className="suggestion-chip" onClick={() =>
sendMessage(s)} style={{
                      padding: "6px 12px",
borderRadius: 20,
                      border: "1.5px solid
#DDD6FE",
                      background: "white",
color: "#6C63FF",
                      fontSize: 12, cursor:
"pointer",
                      transition: "all 0.15s
ease",
                      fontFamily: "inherit",
                    }}>
                      {s}
                    </button>
                ))}
            </div>
          </div>
        )

```

```

    })

    <div ref={bottomRef} />
  </div>

  { /* Input */ }
  <div style={{
    padding: "12px 16px 16px",
    borderTop: "1px solid #F3F2FF",
    background: "white",
  }}>
    <div style={{
      display: "flex", alignItems:
"center", gap: 8,
      background: "#F9F8FF",
      borderRadius: 28,
      border: "1.5px solid #E5E3FF",
      padding: "6px 6px 6px 16px",
      transition: "border-color
0.15s",
    }}>
      <input
        ref={inputRef}
        className="chat-input"
        value={input}
        onChange={e =>
setInput(e.target.value)}
        onKeyDown={handleKey}
        placeholder="Type a
message..."
        disabled={loading}
        style={{
          flex: 1, border: "none",
background: "transparent",

```

```

        fontSize: 14, color:
"#1a1a2e",
        fontFamily: "inherit",
    }}
/>
<button
    className="send-btn"
    onClick={() => sendMessage()}
    disabled={!input.trim() ||
loading}
    style={{
        width: 36, height: 36,
borderRadius: "50%",
        background: input.trim() &&
!loading
            ? "linear-
gradient(135deg, #6C63FF, #8B5CF6)"
            : "#E5E3FF",
        border: "none", cursor:
input.trim() && !loading ? "pointer" :
"default",
        display: "flex",
alignItems: "center", justifyContent:
"center",
        transition: "all 0.15s
ease",
        flexShrink: 0,
    }}
>
    <svg width="16" height="16"
viewBox="0 0 24 24" fill="none">
        <path d="M22 2L11 13"
stroke={input.trim() && !loading ? "white"
: "#A78BFA"} strokeWidth="2"

```

```
strokeLinecap="round" />
      <path d="M22 2L15 22L11
13L2 9L22 22" stroke={input.trim() &&
!loading ? "white" : "#A78BFA"}
strokeWidth="2" strokeLinecap="round"
strokeLinejoin="round" />
    </svg>
  </button>
</div>
<div style={{ textAlign:
"center", fontSize: 11, color: "#C4B5FD",
marginTop: 8 }}>
    Powered by Claude · Press Enter
to send
  </div>
</div>
</div>
</div>
);
}
```